Analysis of GitHub Trending Languages (Short-Term Trends)

Course:

Big Data Engineering, FH Technikum Wien, Summer Semester 2025

Team Members:

- Manpreet Misson
- Timothy Gregorian
- Omar Sidi Mammar

Notebook Description

This notebook presents a streaming pipeline that analyzes short-term technology trends on GitHub using **Apache Kafka** and **Apache Spark Structured Streaming**.

A cleaned JSON dataset of GitHub trending repositories is streamed via Kafka at short intervals. Spark processes the data in real time, aggregates programming language usage, and identifies the **top trending languages**.

The analysis covers a **brief timeframe in 2025** (May 14 – June 18), offering insights into current open-source activity.

The pipeline showcases a **scalable, event-driven architecture** for real-time tech trend analytics.

Goals of this Notebook

- Stream a cleaned dataset of GitHub repositories via **Kafka**.
- Process the stream using **Spark Structured Streaming**.
- Count repositories per programming language and year.
- Visualize the most popular languages in **2025**.

Import Dependencies for Analysis

This block loads all the necessary libraries used for Kafka message production, structured streaming with PySpark, logging, data transformation, and visualization.

In [1]:
```
from kafka import KafkaProducer
import json, time, os
from pyspark.sql import SparkSession
from pyspark.sql.functions import from_json, col, desc, to_timestamp, current_ti
from pyspark.sql.functions import to_timestamp, to_date, year, col, min as min_,
from pyspark.sql.types import StructType, StringType
import logging
```

```
import matplotlib.pyplot as plt
import pandas as pd
import matplotlib.cm as cm
import numpy as np

logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s - %(levelname)s - %(message)s"
)

logger = logging.getLogger("KafkaProducer")
```

Initialize Kafka Producer

This block creates a Kafka producer instance that connects to the specified Kafka broker. It serializes Python dictionaries into JSON format and encodes them as UTF-8 bytes before sending.

In [2]:
```
producer = KafkaProducer(
    bootstrap_servers="172.29.16.101:9092",
    value_serializer=lambda v: json.dumps(v).encode("utf-8")
)
```

```
2025-06-23 10:29:25,922 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.2
9.16.101:9092 <connecting> [IPv4 ('172.29.16.101', 9092)]>: connecting to 172.29.
16.101:9092 [('172.29.16.101', 9092) IPv4]
2025-06-23 10:29:25,925 - INFO - Probing node bootstrap-0 broker version
2025-06-23 10:29:25,927 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.2
9.16.101:9092 <connecting> [IPv4 ('172.29.16.101', 9092)]>: Connection complete.
2025-06-23 10:29:26,035 - INFO - Broker version identified as 2.5.0
2025-06-23 10:29:26,037 - INFO - Set configuration api_version=(2, 5, 0) to skip
auto check_version requests on startup
```

Send Scraped JSON Data to Kafka Topic

This block reads cleaned GitHub trending data from a local JSON file and sends each valid entry to the Kafka topic `"github-trending-all"`. Only entries with a valid `"language"` field are sent. Each message is serialized and pushed using the Kafka producer with a short delay to simulate streaming.

In [3]:
```
topic = "github-trending-all-v3"
json_path = "./ScrapedData/ScrapData_All_Cleaned.json"

with open(json_path, "r", encoding="utf-8") as f:
    data = json.load(f)
    for entry in data:
        if "language" in entry and entry["language"] != "N/A":
            producer.send(topic, entry)
            logger.info(f"Sent: {entry['name']}")
            time.sleep(0.01)

producer.flush()
producer.close()
logger.info("Kafka Producer finished sending.")
```

```
2025-06-23 10:29:26,066 - INFO - <BrokerConnection node_id=1 host=172.29.16.101:9
092 <connecting> [IPv4 ('172.29.16.101', 9092)]>: connecting to 172.29.16.101:909
2 [('172.29.16.101', 9092) IPv4]
2025-06-23 10:29:26,066 - INFO - Sent: microsoft/BitNet
2025-06-23 10:29:26,068 - INFO - <BrokerConnection node_id=1 host=172.29.16.101:9
092 <connecting> [IPv4 ('172.29.16.101', 9092)]>: Connection complete.
2025-06-23 10:29:26,071 - INFO - <BrokerConnection node_id=bootstrap-0 host=172.2
9.16.101:9092 <connected> [IPv4 ('172.29.16.101', 9092)]>: Closing connection.
2025-06-23 10:29:26,080 - INFO - Sent: xaoyaoo/PyWxDump
2025-06-23 10:29:26,092 - INFO - Sent: ahmedkhaleel2004/gitdiagram
2025-06-23 10:29:26,104 - INFO - Sent: i-am-alice/3rd-devs
2025-06-23 10:29:26,117 - INFO - Sent: xming521/WeClone
2025-06-23 10:29:26,132 - INFO - Sent: openai/simple-evals
2025-06-23 10:29:26,145 - INFO - Sent: alibaba/spring-ai-alibaba
2025-06-23 10:29:26,161 - INFO - Sent: airweave-ai/airweave
2025-06-23 10:29:26,173 - INFO - Sent: harry0703/MoneyPrinterTurbo
2025-06-23 10:29:26,187 - INFO - Sent: mem0ai/mem0
2025-06-23 10:29:26,201 - INFO - Sent: overleaf/overleaf
2025-06-23 10:29:26,215 - INFO - Sent: mikumifa/biliTickerBuy
2025-06-23 10:29:26,230 - INFO - Sent: happycola233/tchMaterial-parser
2025-06-23 10:29:26,243 - INFO - Sent: trycua/cua
2025-06-23 10:29:26,257 - INFO - Sent: n8n-io/n8n
2025-06-23 10:29:26,272 - INFO - Sent: Capsize-Games/airunner
2025-06-23 10:29:26,285 - INFO - Sent: facebookresearch/fairchem
2025-06-23 10:29:26,300 - INFO - Sent: neondatabase/neon
2025-06-23 10:29:26,314 - INFO - Sent: google/perfetto
2025-06-23 10:29:26,328 - INFO - Sent: virattt/ai-hedge-fund
2025-06-23 10:29:26,341 - INFO - Sent: XTLS/Xray-core
2025-06-23 10:29:26,355 - INFO - Sent: public-apis/public-apis
2025-06-23 10:29:26,369 - INFO - Sent: CopilotKit/CopilotKit
2025-06-23 10:29:26,382 - INFO - Sent: th-ch/youtube-music
2025-06-23 10:29:26,397 - INFO - Sent: Stirling-Tools/Stirling-PDF
2025-06-23 10:29:26,411 - INFO - Sent: f/awesome-chatgpt-prompts
2025-06-23 10:29:26,425 - INFO - Sent: kortix-ai/suna
2025-06-23 10:29:26,440 - INFO - Sent: zen-browser/desktop
2025-06-23 10:29:26,455 - INFO - Sent: sherlock-project/sherlock
2025-06-23 10:29:26,471 - INFO - Sent: tinygrad/tinygrad
2025-06-23 10:29:26,485 - INFO - Sent: ventoy/Ventoy
2025-06-23 10:29:26,499 - INFO - Sent: facebook/pyrefly
2025-06-23 10:29:26,513 - INFO - Sent: antirez/kilo
2025-06-23 10:29:26,527 - INFO - Sent: appwrite/appwrite
2025-06-23 10:29:26,541 - INFO - Sent: nektos/act
2025-06-23 10:29:26,555 - INFO - Sent: HeyPuter/puter
2025-06-23 10:29:26,569 - INFO - Sent: usememos/memos
2025-06-23 10:29:26,584 - INFO - Sent: microsoft/WSL2-Linux-Kernel
2025-06-23 10:29:26,598 - INFO - Sent: Cysharp/ZLinq
2025-06-23 10:29:26,613 - INFO - Sent: microsoft/semantic-kernel
2025-06-23 10:29:26,628 - INFO - Sent: microsoft/PowerToys
2025-06-23 10:29:26,642 - INFO - Sent: microsoft/WSL
2025-06-23 10:29:26,655 - INFO - Sent: XiaoYouChR/Ghost-Downloader-3
2025-06-23 10:29:26,669 - INFO - Sent: flutter/flutter
2025-06-23 10:29:26,682 - INFO - Sent: freeCodeCamp/freeCodeCamp
2025-06-23 10:29:26,696 - INFO - Sent: microsoft/vscode
2025-06-23 10:29:26,709 - INFO - Sent: modelcontextprotocol/registry
2025-06-23 10:29:26,722 - INFO - Sent: All-Hands-AI/OpenHands
2025-06-23 10:29:26,736 - INFO - Sent: huggingface/huggingface.js
2025-06-23 10:29:26,749 - INFO - Sent: disposable-email-domains/disposable-email-
domains
2025-06-23 10:29:26,763 - INFO - Sent: usebruno/bruno
2025-06-23 10:29:26,777 - INFO - Sent: Fosowl/agenticSeek
```

```
2025-06-23 10:29:26,792 - INFO - Sent: mindsdb/mindsdb
2025-06-23 10:29:26,806 - INFO - Sent: shadps4-emu/shadPS4
2025-06-23 10:29:26,821 - INFO - Sent: groupultra/telegram-search
2025-06-23 10:29:26,834 - INFO - Sent: duixcom/Duix.Heygem
2025-06-23 10:29:26,848 - INFO - Sent: reactos/reactos
2025-06-23 10:29:26,862 - INFO - Sent: kamranahmedse/developer-roadmap
2025-06-23 10:29:26,875 - INFO - Sent: jellyfin/jellyfin
2025-06-23 10:29:26,890 - INFO - Sent: microsoft/qlib
2025-06-23 10:29:26,905 - INFO - Sent: hiyouga/LLaMA-Factory
2025-06-23 10:29:26,919 - INFO - Sent: microsoft/typescript-go
2025-06-23 10:29:26,935 - INFO - Sent: lobehub/lobe-chat
2025-06-23 10:29:26,949 - INFO - Sent: comfyanonymous/ComfyUI
2025-06-23 10:29:26,962 - INFO - Sent: ziglang/zig
2025-06-23 10:29:26,976 - INFO - Sent: langflow-ai/langflow
2025-06-23 10:29:26,990 - INFO - Sent: iptv-org/iptv
2025-06-23 10:29:27,004 - INFO - Sent: uutils/coreutils
2025-06-23 10:29:27,018 - INFO - Sent: microsoft/RD-Agent
2025-06-23 10:29:27,031 - INFO - Sent: pathwaycom/pathway
2025-06-23 10:29:27,043 - INFO - Sent: slowlydev/f1-dash
2025-06-23 10:29:27,056 - INFO - Sent: wg-easy/wg-easy
2025-06-23 10:29:27,070 - INFO - Sent: 78/xiaozhi-esp32
2025-06-23 10:29:27,083 - INFO - Sent: NomicFoundation/hardhat
2025-06-23 10:29:27,097 - INFO - Sent: actions/runner-images
2025-06-23 10:29:27,111 - INFO - Sent: facebook/react-native
2025-06-23 10:29:27,128 - INFO - Sent: zhayujie/chatgpt-on-wechat
2025-06-23 10:29:27,142 - INFO - Sent: duixcom/Duix.mobile
2025-06-23 10:29:27,156 - INFO - Sent: sktime/sktime
2025-06-23 10:29:27,169 - INFO - Sent: labring/FastGPT
2025-06-23 10:29:27,182 - INFO - Sent: Lagrange-Labs/deep-prove
2025-06-23 10:29:27,195 - INFO - Sent: ccxt/ccxt
2025-06-23 10:29:27,209 - INFO - Sent: KwaiVGI/LivePortrait
2025-06-23 10:29:27,222 - INFO - Sent: aaPanel/BillionMail
2025-06-23 10:29:27,237 - INFO - Sent: getzep/graphiti
2025-06-23 10:29:27,251 - INFO - Sent: coleam00/local-ai-packaged
2025-06-23 10:29:27,265 - INFO - Sent: syncthing/syncthing
2025-06-23 10:29:27,280 - INFO - Sent: assimp/assimp
2025-06-23 10:29:27,294 - INFO - Sent: mui/mui-x
2025-06-23 10:29:27,307 - INFO - Sent: googleapis/go-genai
2025-06-23 10:29:27,321 - INFO - Sent: ok-oldking/ok-wuthering-waves
2025-06-23 10:29:27,335 - INFO - Sent: MODSetter/SurfSense
2025-06-23 10:29:27,350 - INFO - Sent: termux/termux-app
2025-06-23 10:29:27,362 - INFO - Sent: MervinPraison/PraisonAI
2025-06-23 10:29:27,376 - INFO - Sent: black-forest-labs/flux
2025-06-23 10:29:27,390 - INFO - Sent: yeongpin/cursor-free-vip
2025-06-23 10:29:27,403 - INFO - Sent: WordPress/gutenberg
2025-06-23 10:29:27,419 - INFO - Sent: onlook-dev/onlook
2025-06-23 10:29:27,432 - INFO - Sent: nautechsystems/nautilus_trader
2025-06-23 10:29:27,447 - INFO - Sent: frdel/agent-zero
2025-06-23 10:29:27,463 - INFO - Sent: donnemartin/system-design-primer
2025-06-23 10:29:27,477 - INFO - Sent: elebumm/RedditVideoMakerBot
2025-06-23 10:29:27,491 - INFO - Sent: gitroomhq/postiz-app
2025-06-23 10:29:27,505 - INFO - Sent: plankanban/planka
2025-06-23 10:29:27,518 - INFO - Sent: netbirdio/netbird
2025-06-23 10:29:27,532 - INFO - Sent: iamgio/quarkdown
2025-06-23 10:29:27,545 - INFO - Sent: ArduPilot/ardupilot
2025-06-23 10:29:27,559 - INFO - Sent: lastmile-ai/mcp-agent
2025-06-23 10:29:27,573 - INFO - Sent: modelcontextprotocol/ruby-sdk
2025-06-23 10:29:27,585 - INFO - Sent: topoteretes/cognee
2025-06-23 10:29:27,600 - INFO - Sent: scrapy/scrapy
2025-06-23 10:29:27,618 - INFO - Sent: coleam00/Archon
```

```
2025-06-23 10:29:27,631 - INFO - Sent: rustdesk/rustdesk
2025-06-23 10:29:27,646 - INFO - Sent: stanfordnlp/dspy
2025-06-23 10:29:27,659 - INFO - Sent: deepsense-ai/ragbits
2025-06-23 10:29:27,671 - INFO - Sent: iib0011/omni-tools
2025-06-23 10:29:27,684 - INFO - Sent: langgenius/dify
2025-06-23 10:29:27,697 - INFO - Sent: codexu/note-gen
2025-06-23 10:29:27,711 - INFO - Sent: tensorzero/tensorzero
2025-06-23 10:29:27,725 - INFO - Sent: jwohlwend/boltz
2025-06-23 10:29:27,738 - INFO - Sent: juspay/hyperswitch
2025-06-23 10:29:27,752 - INFO - Sent: YaLTeR/niri
2025-06-23 10:29:27,765 - INFO - Sent: confident-ai/deepeval
2025-06-23 10:29:27,779 - INFO - Sent: jdepoix/youtube-transcript-api
2025-06-23 10:29:27,794 - INFO - Sent: xiangechen/chili3d
2025-06-23 10:29:27,808 - INFO - Sent: Shubhamsaboo/awesome-llm-apps
2025-06-23 10:29:27,822 - INFO - Sent: infiniflow/ragflow
2025-06-23 10:29:27,835 - INFO - Sent: continuedev/continue
2025-06-23 10:29:27,849 - INFO - Sent: menloresearch/jan
2025-06-23 10:29:27,863 - INFO - Sent: automatisch/automatisch
2025-06-23 10:29:27,876 - INFO - Sent: cloudflare/ai
2025-06-23 10:29:27,890 - INFO - Sent: php/frankenphp
2025-06-23 10:29:27,904 - INFO - Sent: ollama/ollama
2025-06-23 10:29:27,918 - INFO - Sent: vllm-project/vllm
2025-06-23 10:29:27,932 - INFO - Sent: espressif/esp-idf
2025-06-23 10:29:27,948 - INFO - Sent: MystenLabs/sui
2025-06-23 10:29:27,963 - INFO - Sent: firebase/genkit
2025-06-23 10:29:27,976 - INFO - Sent: dail8859/NotepadNext
2025-06-23 10:29:27,993 - INFO - Closing the Kafka producer with 9223372036.0 sec
s timeout.
2025-06-23 10:29:27,996 - INFO - <BrokerConnection node_id=1 host=172.29.16.101:9
092 <connected> [IPv4 ('172.29.16.101', 9092)]>: Closing connection.
2025-06-23 10:29:27,998 - INFO - Kafka Producer finished sending.
```

Initialize Spark Session

This block creates a SparkSession configured to connect to a remote Spark cluster. The application is named `"GitHubTrendingAllTimev1"` and uses the specified Spark master node to distribute tasks.

In [4]:
```
spark = SparkSession.builder \
    .appName("GitHubTrendingAllTime") \
    .master("local[*]") \
    .getOrCreate()
```

```
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel
(newLevel).
25/06/23 10:29:29 WARN NativeCodeLoader: Unable to load native-hadoop library for
your platform... using builtin-java classes where applicable
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4040. Atte
mpting port 4041.
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4041. Atte
mpting port 4042.
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4042. Atte
mpting port 4043.
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4043. Atte
mpting port 4044.
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4044. Atte
mpting port 4045.
25/06/23 10:29:30 WARN Utils: Service 'SparkUI' could not bind on port 4045. Atte
mpting port 4046.
```

Read Streaming Data from Kafka

This block initializes a streaming DataFrame by connecting to a Kafka topic named `"github-trending-all"`. It reads all available data from the earliest offset and listens for new incoming records continuously.

In [5]:
```
df_kafka = spark.readStream \
    .format("kafka") \
    .option("kafka.bootstrap.servers", "172.29.16.101:9092") \
    .option("subscribe", "github-trending-all-v3") \
    .option("startingOffsets", "earliest") \
    .load()
```

```
25/06/23 10:29:31 WARN FileSystem: Cannot load filesystem: java.util.ServiceConfi
gurationError: org.apache.hadoop.fs.FileSystem: com.google.cloud.hadoop.fs.gcs.Go
ogleHadoopFileSystem Unable to get public no-arg constructor
25/06/23 10:29:31 WARN FileSystem: java.lang.NoClassDefFoundError: com/google/ap
i/client/http/HttpRequestInitializer
25/06/23 10:29:31 WARN FileSystem: java.lang.ClassNotFoundException: com.google.a
pi.client.http.HttpRequestInitializer
```

Define Data Schema

This block defines the schema for the incoming JSON data, specifying the expected fields such as repository name, description, programming language, star count, URL, and the timestamp when the data was scraped. This schema ensures that the data is correctly parsed and typed when loaded into Spark.

In [6]:
```
schema = StructType() \
    .add("name", StringType()) \
    .add("description", StringType()) \
    .add("language", StringType()) \
    .add("stars", StringType()) \
    .add("url", StringType()) \
    .add("scraped_at", StringType())
```

Parse and Filter Kafka Data

This code reads the raw Kafka messages as strings, parses them into structured data using the predefined schema, and selects all fields from the parsed JSON. It then filters out entries where the programming language is missing or marked as "N/A" to ensure only valid language data is processed.

In [7]:
```
df = df_kafka.selectExpr("CAST(value AS STRING)") \
    .select(from_json(col("value"), schema).alias("data")) \
    .select("data.*") \
    .filter(col("language").isNotNull() & (col("language") != "N/A"))
```

Extract and Transform Timestamp Columns

Here, the code converts the `scraped_at` string field into a timestamp and a date format, creating new columns `scraped_at_ts` and `scraped_date`. It also extracts the year from the timestamp into a separate `year` column. These transformations facilitate time-based grouping and analysis.

In [8]:
```
df_with_time = df.withColumn("scraped_at_ts", to_timestamp("scraped_at")) \
                 .withColumn("scraped_date", to_date("scraped_at")) \
                 .withColumn("year", year(col("scraped_at_ts")))
```

Start Streaming Query to Memory

This block initiates a streaming query that writes the processed data (`df_with_time`) into an in-memory table named `df_with_time_mem`. The query appends new data every 5 seconds. The `time.sleep(15)` call allows the streaming query to run briefly and accumulate data in memory before further processing.

In [9]:
```
query = df_with_time.writeStream \
    .outputMode("append") \
    .format("memory") \
    .queryName("df_with_time_mem") \
    .trigger(processingTime="5 seconds") \
    .start()

time.sleep(15)
```

```
25/06/23 10:29:32 WARN ResolveWriteToStream: Temporary checkpoint location create
d which is deleted normally when the query didn't fail: /tmp/temporary-51be8f24-b
b86-4039-a706-e7a0c267cc74. If it's required to delete it under any circumstance
s, please set spark.sql.streaming.forceDeleteTempCheckpointLocation to true. Impo
rtant to know deleting temp checkpoint folder is best effort.
25/06/23 10:29:32 WARN ResolveWriteToStream: spark.sql.adaptive.enabled is not su
pported in streaming DataFrames/Datasets and will be disabled.
25/06/23 10:29:33 WARN AdminClientConfig: These configurations '[key.deserialize
r, value.deserializer, enable.auto.commit, max.poll.records, auto.offset.reset]'
were supplied but are not used yet.
```

Retrieve Data from Memory and Determine Date Range

This code queries the in-memory streaming table `df_with_time_mem` to create a static DataFrame `df_static`. It then computes the minimum and maximum scraped dates within the dataset to dynamically determine the analysis time range, which is printed for reference.

In [10]:
```
df_static = spark.sql("SELECT * FROM df_with_time_mem")

min_date, max_date = df_static.select(min_("scraped_date"), max_("scraped_date")
print(f"Timeframe: {min_date} bis {max_date}")
```

```
Timeframe: 2025-05-14 bis 2025-06-19
```

Aggregate Repository Counts by Year and Language

This segment groups the static DataFrame by `year` and `language`, counting the number of repositories for each combination. The count column is renamed to `repo_count` for clarity. The resulting DataFrame is registered as a temporary SQL view `lang_stats_by_year` to enable querying and further analysis using Spark SQL.

In [11]:
```
top_languages_by_year = df_static.groupBy("year", "language") \
    .count() \
    .withColumnRenamed("count", "repo_count")

top_languages_by_year.createOrReplaceTempView("lang_stats_by_year")
```

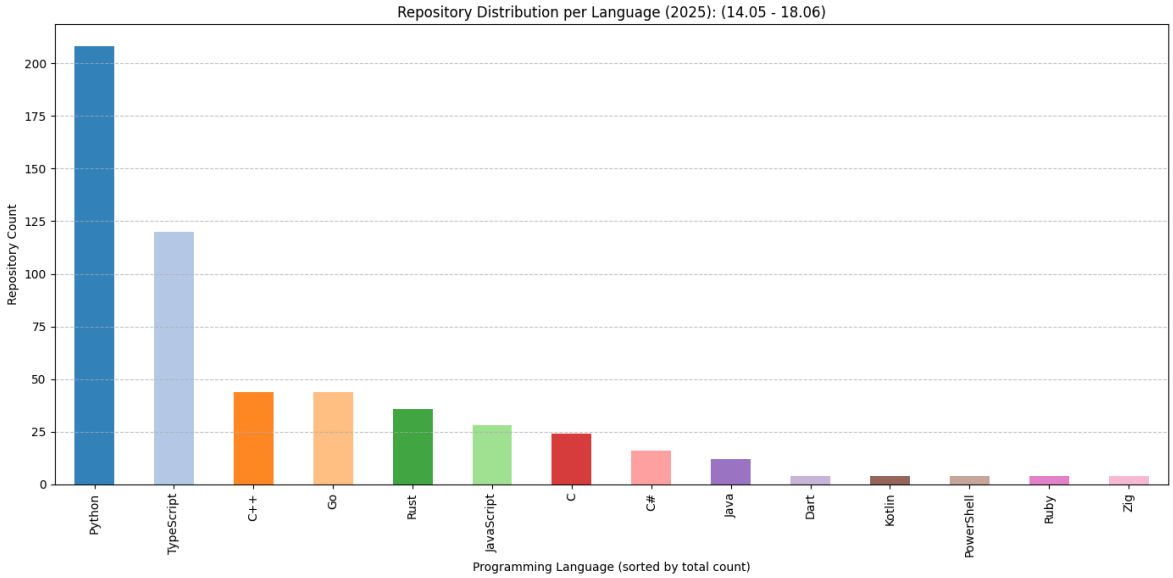
In [12]:
```
df_result = spark.sql("SELECT * FROM lang_stats_by_year ORDER BY year, repo_coun
df_pd = df_result.toPandas()
```

Visualize Language Trends and Totals

Generates a stacked bar chart of repository counts per programming language across years, using a pivoted DataFrame with color mapping and labeled axes. Also prints the total number of repositories and a sorted summary of counts per language.

In [13]:
```
bar_data = df_pd[df_pd["year"] == 2025].groupby("language")["repo_count"].sum().

plt.figure(figsize=(14, 7))

cmap = plt.get_cmap("tab20")
colors = [cmap(i % 20) for i in range(len(bar_data))]

bar_data.plot(
    kind="bar",
    color=colors,
    alpha=0.9,
)

plt.title(f"Repository Distribution per Language (2025): (14.05 - 18.06)")
plt.xlabel("Programming Language (sorted by total count)")
plt.ylabel("Repository Count")
plt.xticks(rotation=90)
plt.grid(True, axis="y", linestyle="--", alpha=0.7)
plt.tight_layout()
plt.show()
```

```
total_repos = df_pd["repo_count"].sum()
print(f"Total Repositories: {total_repos}\n")
for lang, count in zip(df_pd["language"], df_pd["repo_count"]):
    print(f"{lang}: {count} repositories")
```




```
Total Repositories: 552

Python: 208 repositories
TypeScript: 120 repositories
C++: 44 repositories
Go: 44 repositories
Rust: 36 repositories
JavaScript: 28 repositories
C: 24 repositories
C#: 16 repositories
Java: 12 repositories
Kotlin: 4 repositories
Ruby: 4 repositories
Zig: 4 repositories
PowerShell: 4 repositories
Dart: 4 repositories
```

In [14]:
```
query.stop()
logger.info("Spark stream stopped.")
```

```
2025-06-23 10:29:50,108 - INFO - Spark stream stopped.
```

In [15]:
```
spark.stop()
logger.info("Spark session stopped.")
```

```
2025-06-23 10:29:50,442 - INFO - Spark session stopped.
```